Лабораторная работа 1.15. Редактирование и удаление записей в базе данных

Цель лабораторной работы.

Получить практические навыки в разработке для веб-приложения функций редактирования и удаления записей в базе данных.

Введение

Для разработки функционала веб-приложения в части редактирования и удаления записей в базе данных необходимо в репозитории создать ветку dev7 на основе ветки master. После проверки преподавателем работоспособности веб-приложения ветка dev7 должна быть слита с веткой master.

Редактирование страницы списка должностей

Переход к редактированию должностей на странице Должности (role.jsp) реализуется кнопкой «Редактировать». В ссылке этой кнопки необходимо задать значение ссылки href.

```
<td width="20"><a
      href='<c:url value="/editrole?id=${role.getId()}" />'
            role="button" class="btn btn-outline-primary">
            <img alt="Редактировать"
            src="images/icon-edit.png"></a></td>
```

Редактирование должности

Процесс редактирования должности предполагает разработку страницы editrole.jsp и контроллера EditRoleServlet.

Представление страницы «Редактирование должности» приведено на рис. 1.

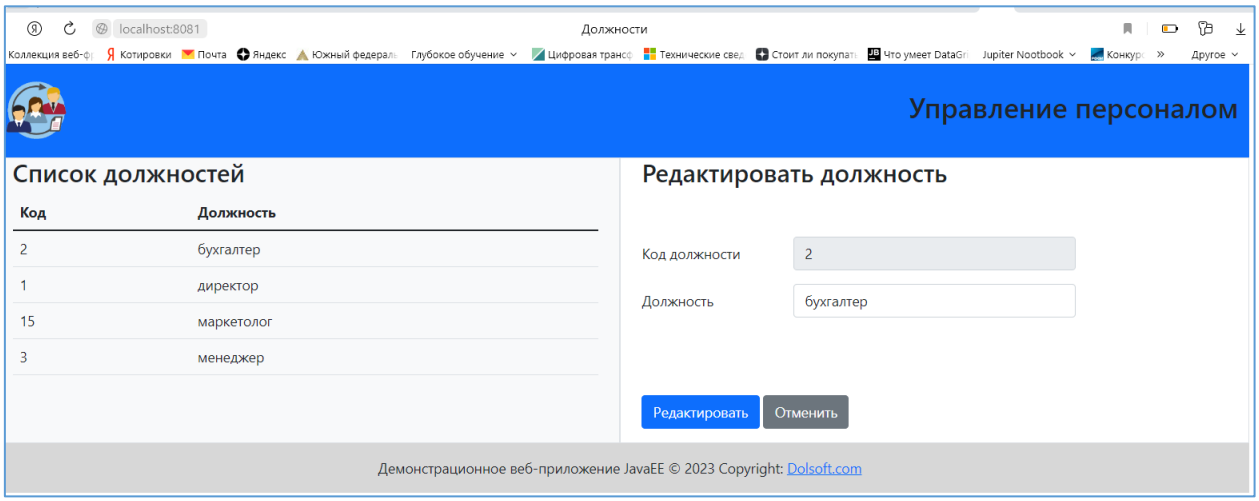


Рисунок 1 - Страница «Редактирование должности»

Код страницы «Редактирование должности» приведен на листинге 1.

Листинг 1.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="jakarta.tags.core"%>
<%@ page import="domain.Role"%>

<!DOCTYPE html>
<html>
<meta http-equiv="Content-Type" content="text/html" charset="UTF-8">
<title>Должности</title>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Roles</title>
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<!-- Bootstrap CSS -->
<link rel="stylesheet" href="css/bootstrap.min.css">
<!-- jQuery -->
<script defer src="js/jquery-3.6.4.js"></script>
<!-- Bootstrap JS + Popper JS -->
<script defer src="js/bootstrap.min.js"></script>
</head>
<body>
    <div class="container-fluid">
    <jsp:include page="/views/header.jsp" />
    <div class="container-fluid">
    <div class="row justify-content-start ">
    <div class="col-6 border bg-light px-4">
    <h3>Список должностей</h3>
    <table class="table">
    <thead>
```

```
        <th scope="col">Код</th>
        <th scope="col">Должность</th>
        </thead>
        <tbody>
        <c:forEach var="role" items="${roles}">
        <tr>
        <td>${role.getId()}</td>
        <td>${role.getNamerole()}</td>
        </tr>
        </c:forEach>
        </tbody>
        </table>
        </div>
        <div class="col-6 border px-4">
        <form method="POST" action="">
        <h3>Редактировать должность</h3>
        <br> <br>
        <div class="mb-3 row">
        <label for="idrole" class="col-sm-3 col-form-label">
             Код должности</label>
        <div class="col-sm-6">
        <input type="text" class="form-control" readonly
        value="${roleEdit.getId()}" />
        </div>
        </div>
        <div class="mb-3 row">
        <br> <label for="inputRole"
             class="col-sm-3 col-form-label">Должность</label>
        <div class="col-sm-6">
        <input type="text" name="inputRole" class="form-control"
        value="${roleEdit.getNamerole()}" id="personRole" />
        </div>
        </div>
        <p>
        <br> <br> <br>
        <button type="submit"
             class="btn btn-primary">Редактировать</button>
        <a href='<c:url value="/role" />' role="button"
             class="btn btn-secondary">Отменить</a>
        <br>
        </p>
        </form>
        </div>
        </div>
        </div>
        <jsp:include page="/views/footer.jsp" />
        </div>
</body>
</html>
```

Представление editrole.jsp во многом аналогично представлению role.jsp.

На странице «Редактирование должности» в отличие от страницы «Должности» в таблице «Список должностей» отсутствуют кнопки «Редактировать» и «Удалить». В области «Редактирование должности» добавлена информация о коде должности, причем значение кода определено как «только для чтения».

```
<div class="mb-3 row">
      <label for="idrole" class="col-sm-3 col-form-label">
            Код должности</label>
      <div class="col-sm-6">
            <input type="text" class="form-control" readonly
            value="${rolesEdit[0].getId()}" />
      </div>
</div>
```

Кнопка «Отменить» используется для отмены редактирования должности и возврата на страницу «Должности».

Код сервлета EditRoleServlet приведен на листинге 2.

Листинг 2.

```
package controller;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.List;

import domain.Role;
import exception.DAOException;
import dao.RoleDbDAO;
import dao.ConnectionProperty;

@WebServlet("/editrole")
public class EditRoleServlet extends HttpServlet {
```

```
	private static final long serialVersionUID = 1L;
	ConnectionProperty prop;

    public EditRoleServlet()
            throws FileNotFoundException, IOException {
        super();
        prop = new ConnectionProperty();
    }


	protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {
            response.setContentType("text/html");
            String userPath;
            List<Role> roles;
            Role editrole = null;
            RoleDbDAO dao = new RoleDbDAO();

            try {
                  roles = dao.findAll();
                  request.setAttribute("roles", roles);
            } catch (DAOException e) {
                  e.printStackTrace();
            }

            String strId = request.getParameter("id");
            Long id = null;      // id редактируемой должности
            if(strId != null) {
                  id = Long.parseLong(strId);
            }

            try {
                  editrole = dao.findById(id);
            } catch (DAOException e) {
                  // TODO Auto-generated catch block
                  e.printStackTrace();
            }

            request.setAttribute("roleEdit", editrole);

            userPath = request.getServletPath();
            if ("/editrole".equals(userPath)) {
                  request.getRequestDispatcher("/views/editrole.jsp")
                        .forward(request, response);
            }
      }


	protected void doPost(HttpServletRequest request,
HttpServletResponse response)
                  throws ServletException, IOException {
```

```
            RoleDbDAO dao = new RoleDbDAO();
            String strId = request.getParameter("id");
            Long id = null;
            if(strId != null) {
                  id = Long.parseLong(strId);
            }
            String namerole = request.getParameter("inputRole");
            Role editrole = new Role(id, namerole);
            try {
                  dao.update(editrole);
            } catch (DAOException e) {
                  e.printStackTrace();
            }

            response.sendRedirect("/persons/role");

      }
}
```

Код сервлета EditRoleServlet основывается на сервлете RoleServlet.

В методе doGet( ) добавлен фрагмент кода, который формирует данные для редактируемой записи и передает параметр rolesEdit с помощью метода setAttribute( ), представлению.

```
String strId = request.getParameter("id");
            Long id = null;     // id редактируемой должности
            if(strId != null) {
                  id = Long.parseLong(strId);
            }

            try {
                  editrole = dao.findById(id);
            } catch (DAOException e) {
                  e.printStackTrace();
            }

            request.setAttribute("roleEdit", editrole);
```

В методе doPost( ) из параметра request получаем параметры id и rolename – наименование должности для создания редактируемого класса editrole.

```
String strId = request.getParameter("id");
Long id = null;
if(strId != null) {
```

```
        id = Long.parseLong(strId);
}
String namerole = request.getParameter("inputRole");
Role editrole = new Role(id, namerole);
```

Затем с помощью методы update(editrole) производится обновление данных в базе данных и перенаправление на страницу role.jsp

```
try {
        editrole = dao.findById(id);
} catch (DAOException e) {
        e.printStackTrace();
}
response.sendRedirect("/persons/role");
```

На рис. 1 показана страница «Редактирование должности», которая открыта для редактирования должности «бухгалтер». Состояние таблицы roles базы данных до редактирования приведено на рис. 2.

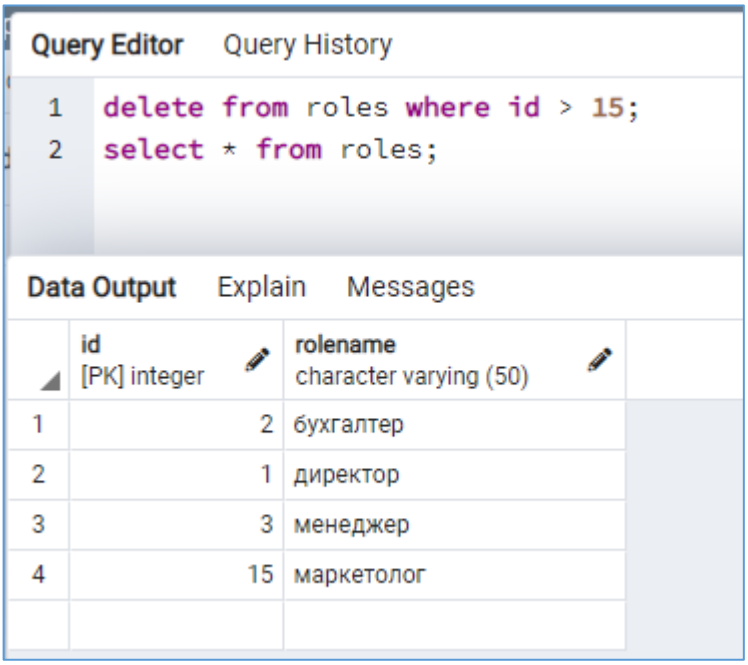


Рисунок 2 - Состояние таблицы roles базы данных до редактирования

Изменим должность «бухгалтер» на должность «главный бухгалтер» (рис. 3).

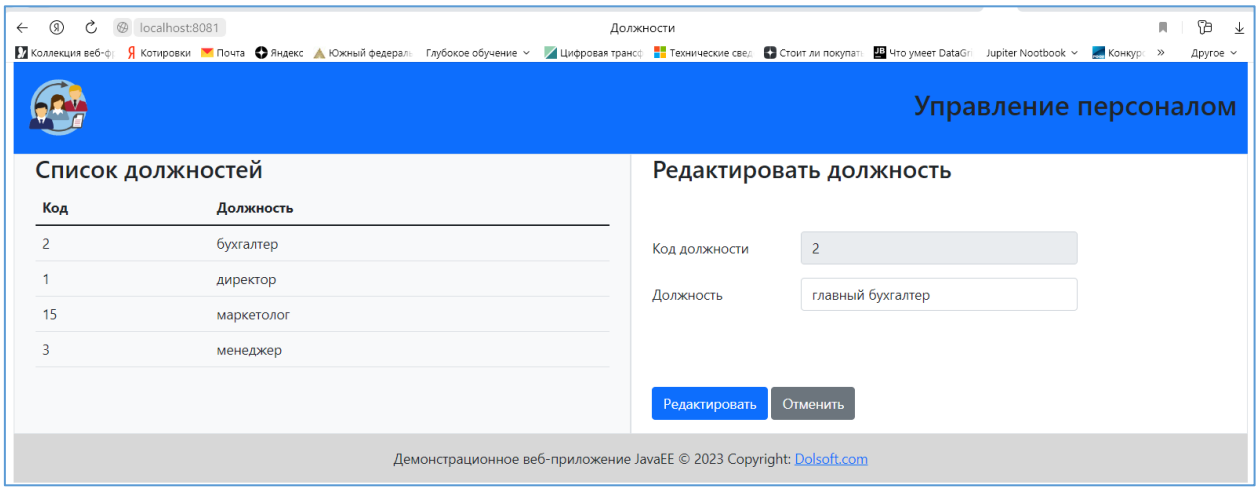


Рисунок 3 – Изменение должности «бухгалтер» на должность «главный бухгалтер»

После нажатия кнопки «Редактировать» в списке должностей отображается отредактированная запись (рис. 4).

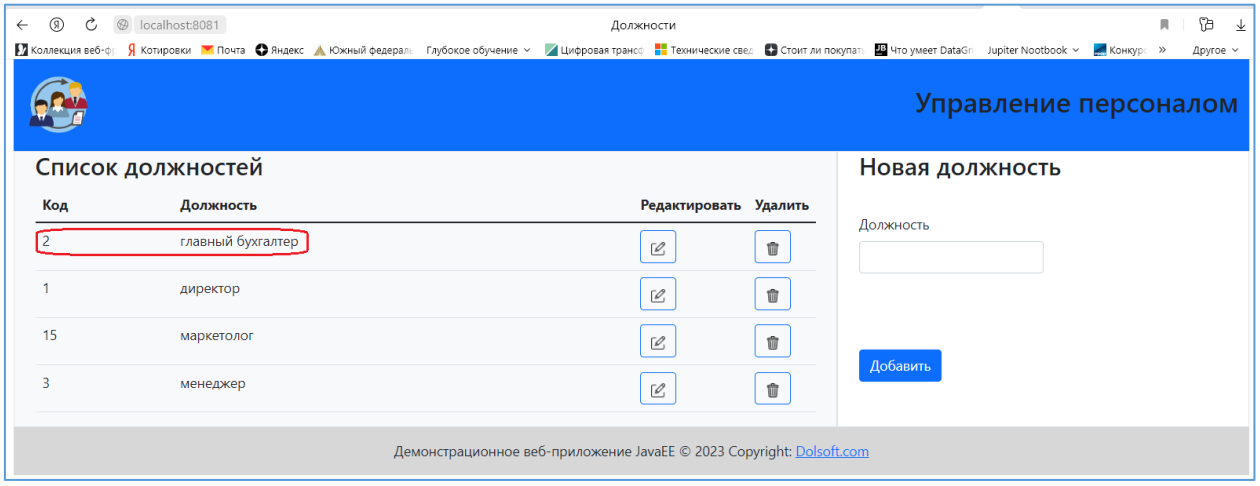


Рисунок 4 – Результаты редактирования должности

В базе данных также отображается результат редактирования записи с id = 2 (рис. 5).

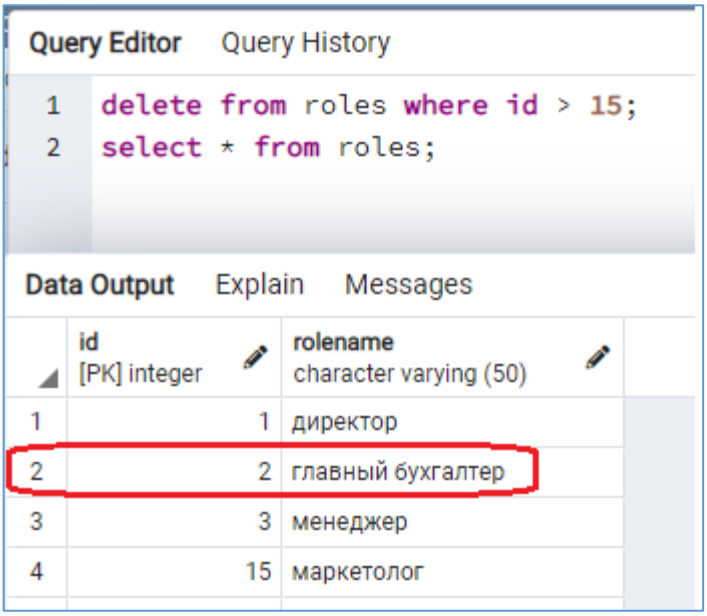


Рисунок 5 - Состояние таблицы roles базы данных после редактирования

Удаление должности

Процесс удаления должности предполагает модификацию страницы role.jsp (Должности) и разработку контроллера DeleteRoleServlet.

На странице role.jsp внесем изменения в описание кнопки «Удалить».

```
<td width="20"><a
      href="<c:url value="/deleterole?id=${role.getId()}" />"
      role="button" class="btn btn-outline-primary">
      <img alt="Удалить" src="images/icon-delete.png"
      onclick="return confirm('Удалить должность с кодом:
'+${role.getId()}+'?')">
</a></td>
```

Во-первых добавим ссылку на конечную точку сервлета DeleteRoleServlet с параметром кода id.

```
href="<c:url value="/deleterole?id=${role.getId()}" />"
```

Для кнопки «Удалить» добавим событие onclick, по которому будет срабатывать метод confirm( ).

```
onclick="return confirm('Удалить должность с кодом:'+
${role.getId()}+'?')"
```

Код сервлета DeleteRoleServlet приведен на листинге 3.

Листинг 3.

```
package controller;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.FileNotFoundException;
import java.io.IOException;

import dao.ConnectionProperty;
import dao.RoleDbDAO;
import exception.DAOException;

@WebServlet( "/deleterole")
public class DeleteRoleServlet extends HttpServlet {

      private static final long serialVersionUID = 1L;
      ConnectionProperty prop;

      public DeleteRoleServlet() throws FileNotFoundException,
IOException {
         super();
         prop = new ConnectionProperty();
    }

      protected void doGet(HttpServletRequest request,
HttpServletResponse response)
                  throws ServletException, IOException {

            RoleDbDAO dao = new RoleDbDAO();

            String strId = request.getParameter("id");
            Long deleteid = null;
            if(strId != null) {
                  deleteid = Long.parseLong(strId);
            }

            try {
                  dao.delete(deleteid);
            } catch (DAOException e) {
                  e.printStackTrace();
            }
            response.sendRedirect("/persons/role");
      }
```

```
}
```

В сервлете используется только метод doGet( ), в котором создается экземпляр dao класса RoleDbDAO, формируется параметр id удаляемой должности из request с помощью метода getParameter( ).

```
String strId = request.getParameter("id");
Long deleteid = null;
if(strId != null) {
	deleteid = Long.parseLong(strId);
}
```

Удаление должности производится методом dao.delete( ).

```
try {
	dao.delete(deleteid);
} catch (DAOException e) {
	e.printStackTrace();
}
```

При завершении метода doGet( ) происходит переадресация на страницу «Должности».

```
response.sendRedirect("/persons/role")
```

На рис. 6 показано состояние списка должностей перед удалением должности «водитель». Состояние таблицы roles базы данных до удаления приведено на рис. 7. После нажатия кнопки «Удалить» в строке с данными по должности «водитель» в браузере выводится диалоговое окно с запросом на подтверждение удаления должности (рис. 8). Пользователь может нажать кнопку «Нет» для отмены удаления или кнопку «Да» для подтверждения удаления выбранной должности.

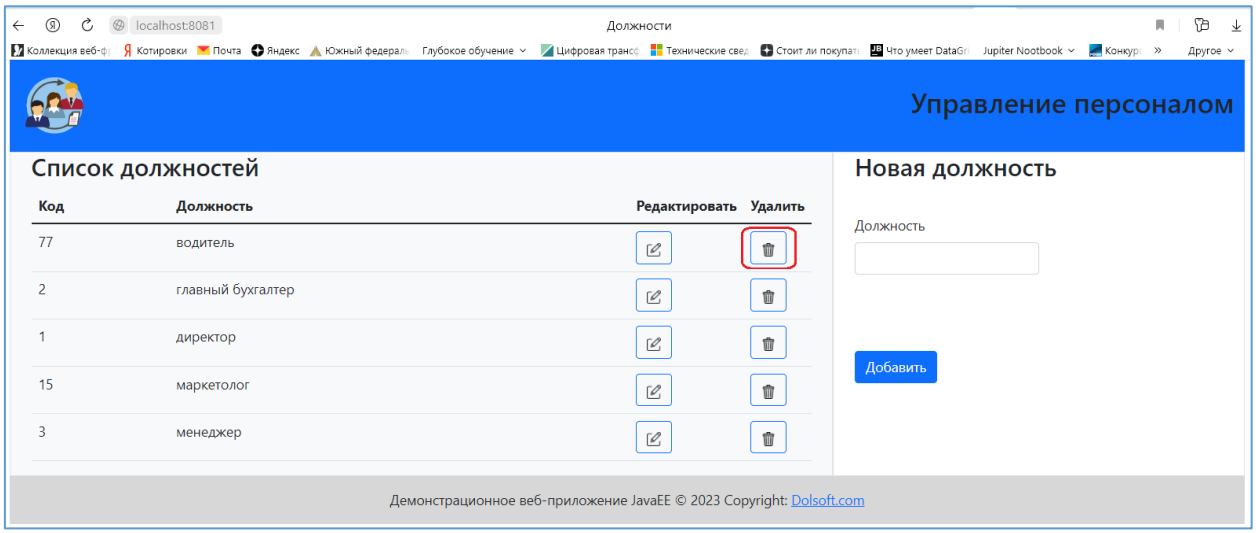


Рисунок 6 – Окно списка должностей до удаления должности «водитель»

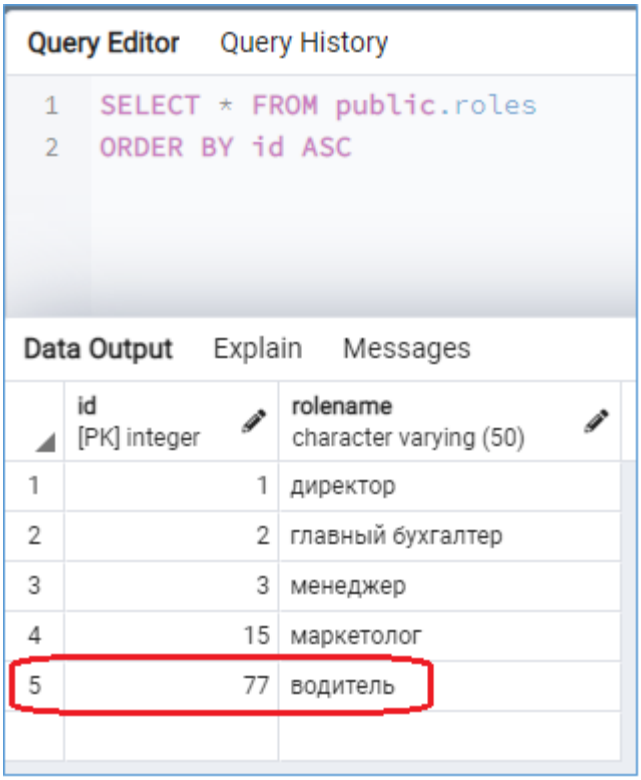


Рисунок 7 – Состояние базы данных до удаления записи

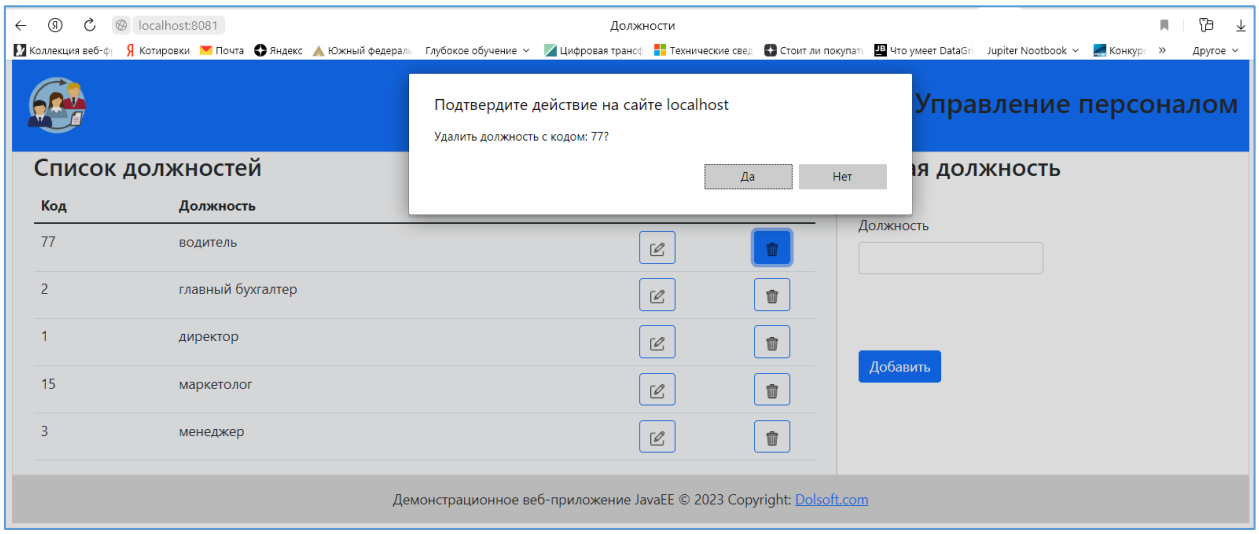


Рисунок 8 – Запрос подтверждения удаления должности

При нажатии кнопки «Да» должность «водитель» будет удалена из базы данных, что отображается в списке должностей (рис. 9).

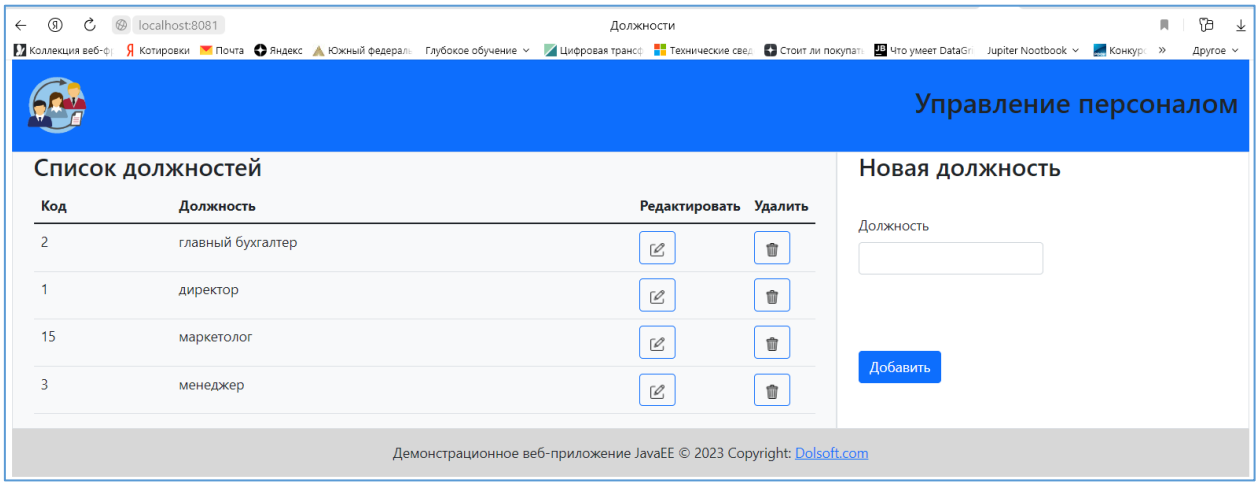


Рисунок 9 – Окно списка должностей после удаления

В базе данных также отображается результат удаления записи с id = 77 (рис. 10).

Data Output | Explain | Messages

	id [PK] integer	rolename character varying (50)
1	1	директор
2	2	главный бухгалтер
3	3	менеджер
4	15	маркетолог

Рисунок 10 – Состояние базы данных после удаления записи

Редактирование страницы «Сотрудники»

Для реализации функций редактирования и удаления данных о сотрудниках из базы данных веб-приложения на странице preson.jsp (Сотрудники) для кнопок «Редактировать» и «Удалить» определим значения ссылок href.

```
<td width="20">
<a href="<c:url value="/editperson?id=${person.getId()}"/>"
      role="button" class="btn btn-outline-primary">
      <img alt="Редактировать" src="images/icon-edit.png">
</a></td>
<td width="20">
<a href="<c:url value="/deleteperson?id=${person.getId()}" />"
      role="button" class="btn btn-outline-primary">
      <img alt="Удалить" src="images/icon-delete.png">
</a></td>
```

При редактировании данных по сотруднику осуществляется переход по пути /editrole?id=${role.getId( ), а при удалении – /deleteperson?id=${person.getId( ).

Редактирование данных по сотрудникам

Процесс редактирования данных по сотрудникам предполагает разработку страницы editperson.jsp и контроллера EditPersonServlet.

Представление страницы «Редактирование данных» при редактировании данных сотрудника с кодом 39 приведено на рис. 11.

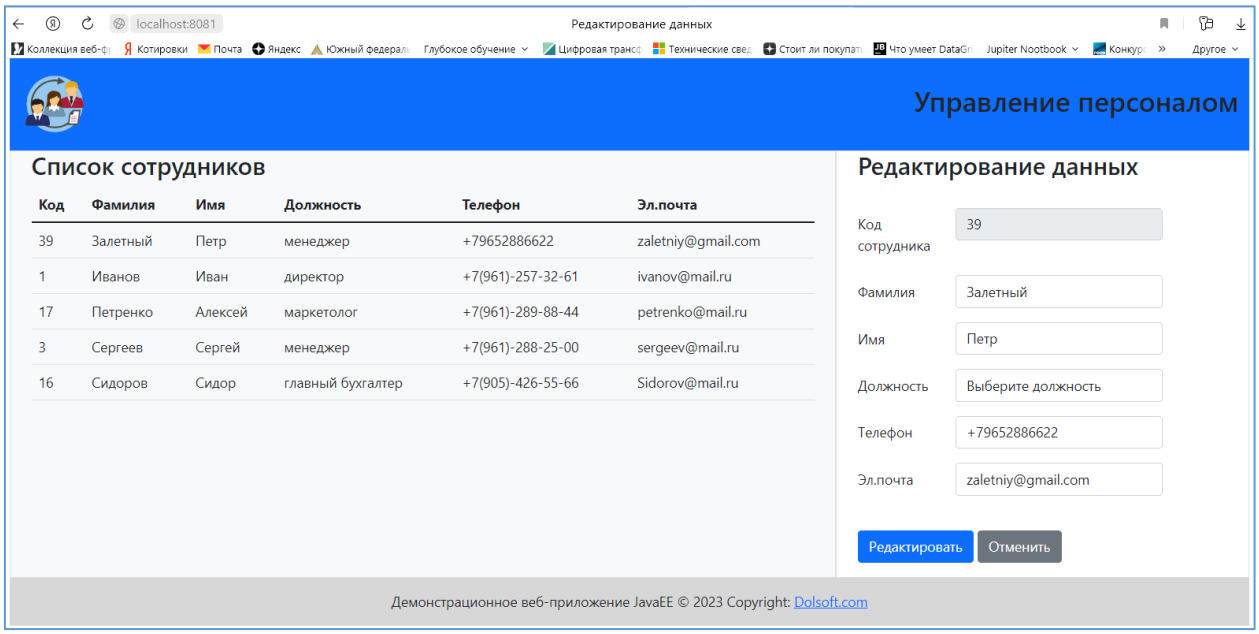


Рисунок 11 - Страница «Редактирование данных»

Код страницы «Редактирование данных» приведен на листинге 6.

Листинг 4.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="jakarta.tags.core"%>

<%@ page import="domain.Role"%>
<%@ page import="domain.Person"%>

<!DOCTYPE html>
<html>
<meta http-equiv="Content-Type" content="text/html" charset="UTF-8">
<title>Редактирование данных</title>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Persons</title>
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<!-- Bootstrap CSS -->
<link rel="stylesheet" href="css/bootstrap.min.css">
<!-- jQuery -->
<script defer src="js/jquery.min.js"></script>
<!-- Bootstrap JS + Popper JS -->
<script defer src="js/bootstrap.min.js"></script>

</head>
<body>
<body>
```

```
<div class="container-fluid">
<jsp:include page="/views/header.jsp" />
<div class="container-fluid">
<div class="row justify-content-start ">
<div class="col-8 border bg-light px-4">
<h3>Список сотрудников</h3>
<table class="table">
<thead>
<th scope="col">Код</th>
<th scope="col">Фамилия</th>
<th scope="col">Имя</th>
<th scope="col">Должность</th>
<th scope="col">Телефон</th>
<th scope="col">Эл.почта</th>
</thead>
<tbody>
<c:forEach var="person" items="${persons}">
<tr>
<td>${person.getId()}</td>
<td>${person.getLastName()}</td>
<td>${person.getFirstName()}</td>
<td>${person.getRole()}</td>
<td>${person.getPhone()}</td>
<td>${person.getEmail()}</td>
</tr>
</c:forEach>
</tbody>
</table>
</div>
<div class="col-4 border px-4">
<form method="POST" action="">
<h3>Редактирование данных</h3>
<br>
<div class="mb-3 row">
<label for="idperson" class="col-sm-3 col-form-label">
     Код сотрудника</label>
<div class="col-sm-7">
<input type="text" class="form-control" readonly
     id="idperson" value="${personEdit.getId()}" />
</div>
</div>
<div class="mb-3 row">
<label for="lastname" class="col-sm-3 col-form-label">
     Фамилия</label>
<div class="col-sm-7">
<input type="text" class="form-control" id="staticLastname"
     name="lastname" value="${personEdit.getLastName()}"/>
</div>
</div>
<div class="mb-3 row">
<label for="firstname" class="col-sm-3 col-form-label">
```

```
            Имя</label>
        <div class="col-sm-7">
        <input type="text" class="form-control" id="staticFirstname"
            name="firstname" value="${personEdit.getFirstName()}"/>
        </div>
        </div>
        <div class="mb-3 row">
        <label for="rolename" class="col-sm-3 col-form-label">
            Должность</label>
        <div class="col-sm-7">
        <select name="role" class="form-control">
        <option>Выберите должность</option>
        <c:forEach var="role" items="${roles}">
        <option value="${role}">
        <c:out value="${role.getNamerole()}"></c:out>
        </option>
        </c:forEach>
        </select>
        </div>
        </div>
        <div class="mb-3 row">
        <label for="phone" class="col-sm-3 col-form-label">
            Телефон</label>
        <div class="col-sm-7">
        <input type="text" class="form-control" id="staticphone"
            name="phone" value="${personEdit.getPhone()}"/>
        </div>
        </div>
        <div class="mb-3 row">
        <label for="email" class="col-sm-3 col-form-label">Эл.почта
        </label>
        <div class="col-sm-7">
        <input type="text" class="form-control" id="staticemail"
            name="email" value="${personEdit.getEmail()}"/>
        </div>
        </div>
        <p>
        <br>
        <button type="submit" class="btn btn-
primary">Редактировать</button>
        <a href='<c:url value="/person" />' role="button"
            class="btn btn-secondary">Отменить</a>
        </p>
        </form>
        </div>
        </div>
        </div>
        <jsp:include page="/views/footer.jsp" />
        </div>
</body>
</html>
```

Изменения, внесенные в страницу editperson.jsp, по отношению к странице person.jsp аналогичные изменениям, проведенным в странице editrole.jsp, по отношению к странице role.jsp.

Код сервлета EditPersovServlet приведен на листинге 5.

Листинг 5.

```
package controller;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.List;

import dao.ConnectionProperty;
import dao.PersonDbDAO;
import dao.RoleDbDAO;
import domain.Person;
import domain.Role;
import exception.DAOException;

@WebServlet("/editperson")
public class EditPersonServlet extends HttpServlet {

      private static final long serialVersionUID = 1L;
      ConnectionProperty prop;

         public EditPersonServlet()
            throws FileNotFoundException, IOException {
         super();
         prop = new ConnectionProperty();
     }

      protected void doGet(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {
            response.setContentType("text/html");
            String userPath;
            List<Role> roles;
            List<Person> persons = null;
            Person editperson = null;
            RoleDbDAO daoRole = new RoleDbDAO();
```

```
            PersonDbDAO dao = new PersonDbDAO();

            try {
                  roles = daoRole.findAll();
                  request.setAttribute("roles", roles);
            } catch (DAOException e) {
                  e.printStackTrace();
            }

            try {
                  persons = dao.findAll();
                  roles = daoRole.findAll();
                  for (Person person: persons) {
      person.setRole(daoRole.FindById(person.getIdRole(), roles));
                  }
            }catch (DAOException e) {
                  e.printStackTrace();
            }

            String strId = request.getParameter("id");
            Long id = null;      // id редактируемого сотрудника
            if(strId != null) {
                  id = Long.parseLong(strId);
            }

            try {
                  editperson = dao.findById(id);
            } catch (DAOException e) {
                  e.printStackTrace();
            }

            request.setAttribute("personEdit", editperson);
            request.setAttribute("persons", persons);

            userPath = request.getServletPath();
            if ("/editperson".equals(userPath)) {

      request.getRequestDispatcher("/views/editperson.jsp").forward(req
uest, response);
            }
      }

      protected void doPost(HttpServletRequest request,
HttpServletResponse response)
                  throws ServletException, IOException {

            PersonDbDAO dao = new PersonDbDAO();
            String strId = request.getParameter("id");
            Long id = null;
            if(strId != null) {
                  id = Long.parseLong(strId);
```

```
            }

            String firstName = request.getParameter("firstname");
            String lastName = request.getParameter("lastname");
            String email = request.getParameter("email");
            String phone = request.getParameter("phone");

            String role =  request.getParameter("role");
            int index1 = role.indexOf('=');
            int index2 = role.indexOf(",");
            String r1 = role.substring(index1+1, index2);
            Long idRole = Long.parseLong(r1.trim());

            Person editPerson = new Person(id, firstName, lastName,
phone, email, idRole);

            try {
                  dao.update(editPerson);
            } catch (DAOException e) {
                  e.printStackTrace();
            }

            response.sendRedirect("/persons/person");
      }

}
```

Метод doGet( ) сервлета не отличается от метода doGet( ) сервлета PersonServlet. Метод doPost( ) также, в основном, аналогичен методу doPost( ) сервлета PersonServlet, за следующими исключениями. Для редактирования данных в базе данных необходим id для данных редактируемого сотрудника.

```
String strId = request.getParameter("id");
            Long id = null;
            if(strId != null) {
                  id = Long.parseLong(strId);
            }
```

После создания экземпляра editPerson класса Person

```
Person editPerson = new Person(id, firstName, lastName, phone, email,
idRole);
```

Редактирование данных выполняется методом update( ).

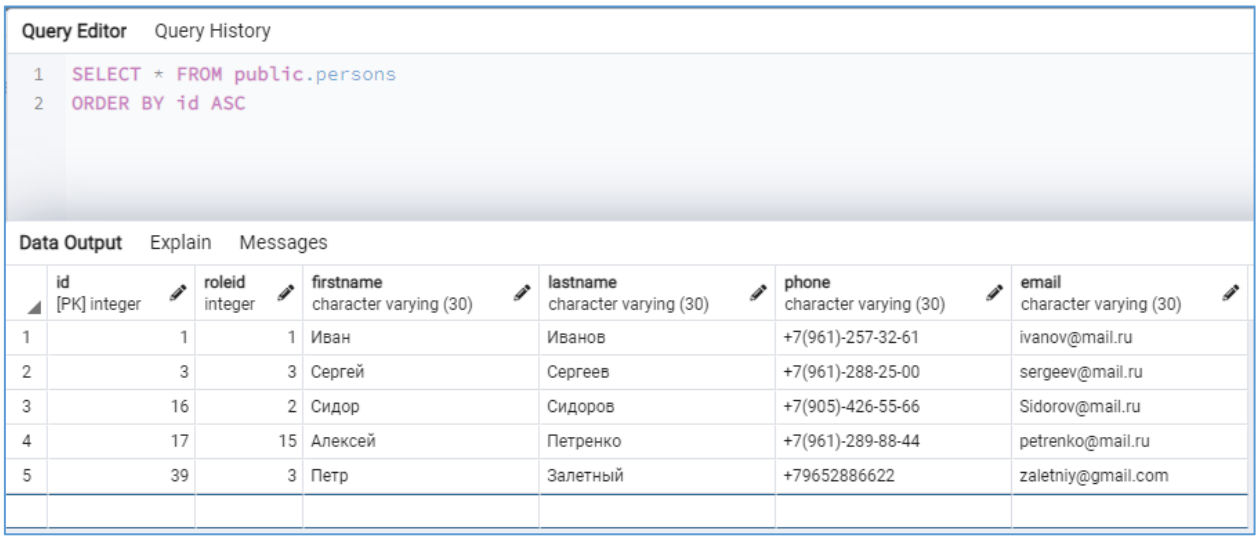
```
try {
```

```
        dao.update(editPerson);
} catch (DAOException e) {
        e.printStackTrace();
}
```

После успешного выполнения операции обновления данных update( ) происходит переадресация на страницу person.jsp (Сотрудники).

```
response.sendRedirect("/persons/person");
```

На рис. 11 показана страница «Редактирование данных», которая открыта для редактирования данных сотрудника Залетный Петр с кодом 37. Состояние таблицы persons базы данных до редактирования приведено на рис. 14.



Query Editor | Query History

```
1  SELECT * FROM public.persons
2  ORDER BY id ASC
```

Data Output | Explain | Messages

	id [PK] integer	roleid integer	firstname character varying (30)	lastname character varying (30)	phone character varying (30)	email character varying (30)
1	1	1	Иван	Иванов	+7(961)-257-32-61	ivanov@mail.ru
2	3	3	Сергей	Сергеев	+7(961)-288-25-00	sergeev@mail.ru
3	16	2	Сидор	Сидоров	+7(905)-426-55-66	Sidorov@mail.ru
4	17	15	Алексей	Петренко	+7(961)-289-88-44	petrenko@mail.ru
5	39	3	Петр	Залетный	+79652886622	zaletniy@gmail.com

Рисунок 12 – Состояние базы данных persons до редактирования

Для демонстрации функционала редактирования данных сотрудника добавим к каждому (кроме поля телефон) полю 1, должность изменим на «водитель», а последние цифры телефона – на 33, при этом изменим формат поля телефон (рис. 13).

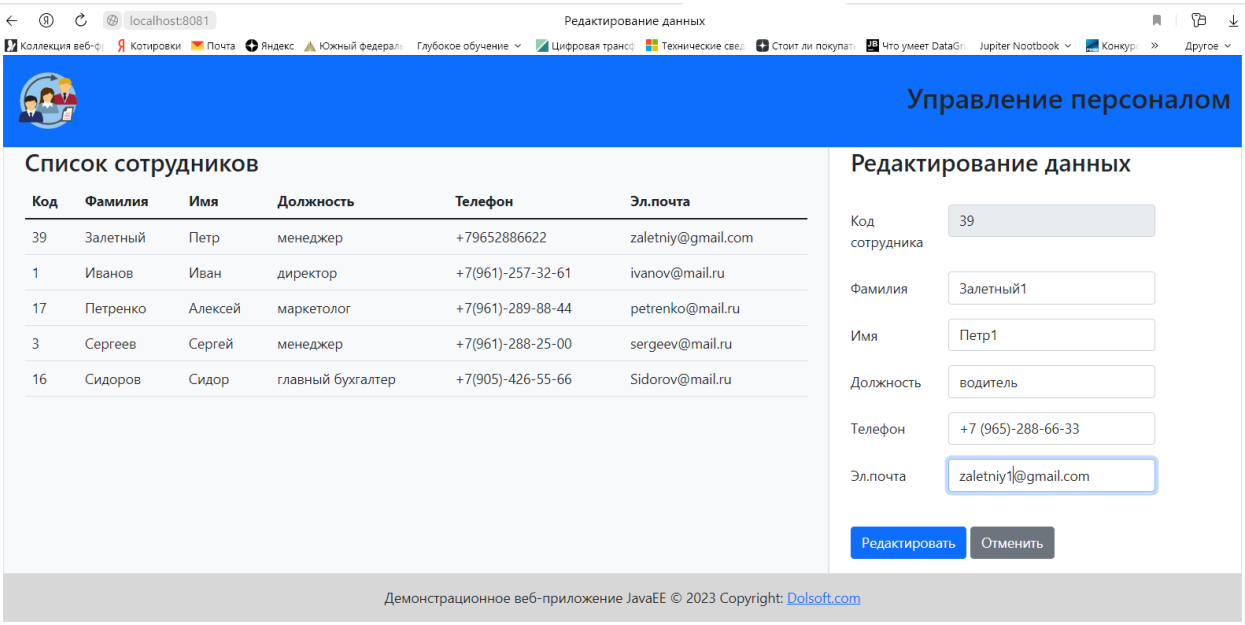


Рисунок 13 – Редактирование данных сотрудника Петр Петров с кодом 39.

После нажатия кнопки «Редактирование» данные по сотруднику будут изменены (рис. 14).

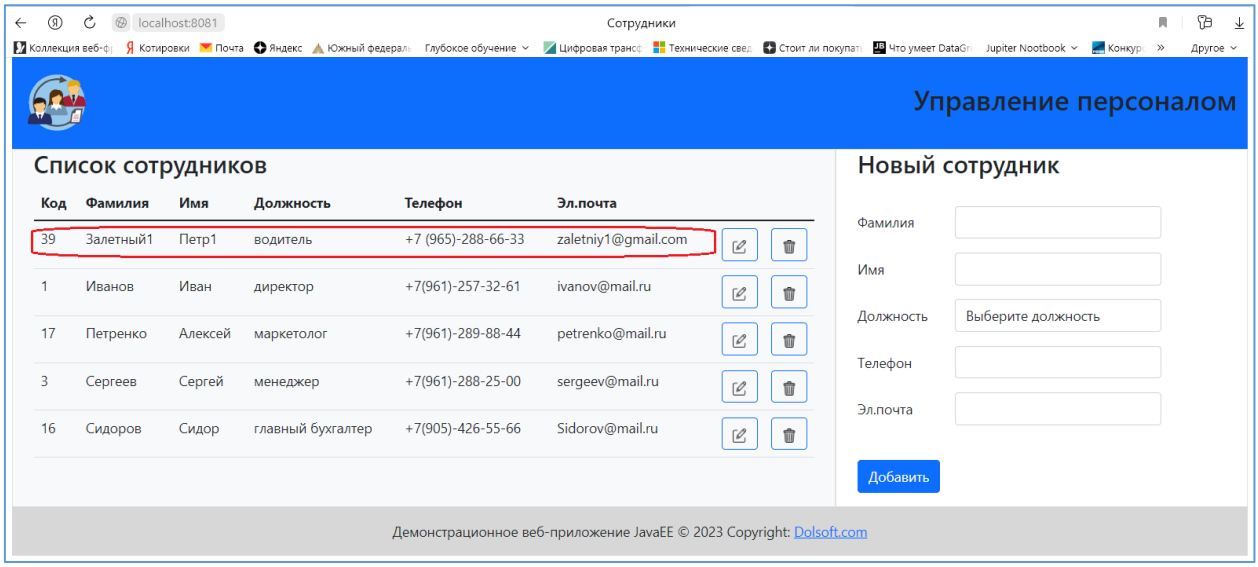
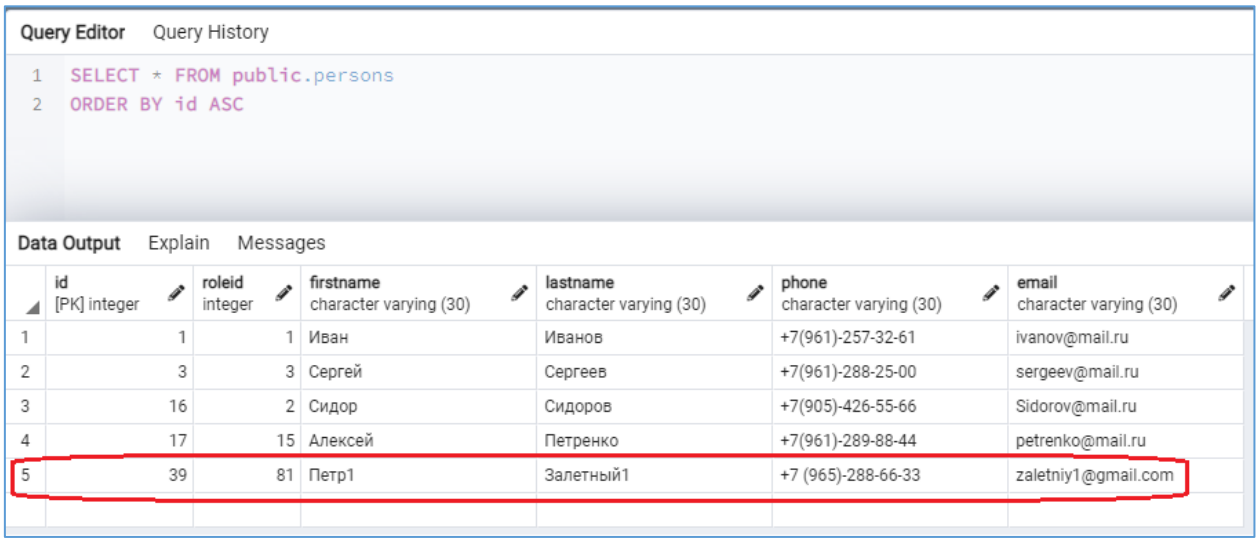


Рисунок 14 – Измененные данные сотрудника Залетный Петр с кодом 39.

Изменения в базе данных persons для таблицы persons приведены на рис. 15



	id [PK] integer	roleid integer	firstname character varying (30)	lastname character varying (30)	phone character varying (30)	email character varying (30)
1	1	1	Иван	Иванов	+7(961)-257-32-61	ivanov@mail.ru
2	3	3	Сергей	Сергеев	+7(961)-288-25-00	sergeev@mail.ru
3	16	2	Сидор	Сидоров	+7(905)-426-55-66	Sidorov@mail.ru
4	17	15	Алексей	Петренко	+7(961)-289-88-44	petrenko@mail.ru
5	39	81	Петр1	Залетный1	+7 (965)-288-66-33	zaletniy1@gmail.com

Рисунок 15 – Состояние базы данных после редактирования данных по сотруднику

Удаление данных по сотруднику

Процесс удаления данных по сотруднику предполагает разработку контроллера DeletePersonServlet.

Предварительно на странице person.jsp необходимо при определении кнопки «Удалить» задать адрес ссылки href и добавить событие onclick/

```
<td width="20">
<a href="<c:url value="/deleteperson?id=${person.getId()}" />"
      role="button" class="btn btn-outline-primary">
      <img alt="Удалить" src="images/icon-delete.png"
      onclick="return confirm('Удалить сотрудника с кодом:
            '+${person.getId()}+'?')">
</a></td>
```

Код сервлета DeletePersonServlet аналогичен коду сервлета DeleteRoleServlet и приведен на листинге 6.

Листинг 6.

```
package controller;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
```

```
import jakarta.servlet.http.HttpServletResponse;

import java.io.FileNotFoundException;
import java.io.IOException;

import dao.ConnectionProperty;
import dao.PersonDbDAO;
import exception.DAOException;

@WebServlet( "/deleteperson")
public class DeletePersonServlet extends HttpServlet {
     private static final long serialVersionUID = 1L;
     ConnectionProperty prop;
    public DeletePersonServlet() throws FileNotFoundException,
IOException {
        super();
        prop = new ConnectionProperty();
    }

     protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {

          PersonDbDAO dao = new PersonDbDAO();
          String strId = request.getParameter("id");
          Long deleteid = null;
          if(strId != null) {
               deleteid = Long.parseLong(strId);
          }

          try {
               dao.delete(deleteid);
          } catch (DAOException e) {
               e.printStackTrace();
          }
          response.sendRedirect("/persons/person");
     }
}
```

На рис. 16 показано состояние списка сотрудников перед удалением сотрудника с кодом 39. Состояние таблицы persons базы данных до удаления приведено на рис. 17. После нажатия кнопки «Удалить» в браузере выводится диалоговое окно с запросом на подтверждение удаления сотрудника (рис. 18). Пользователь может нажать кнопку «Нет» для отмены удаления или кнопку «Да» для подтверждения удаления выбранного сотрудника.

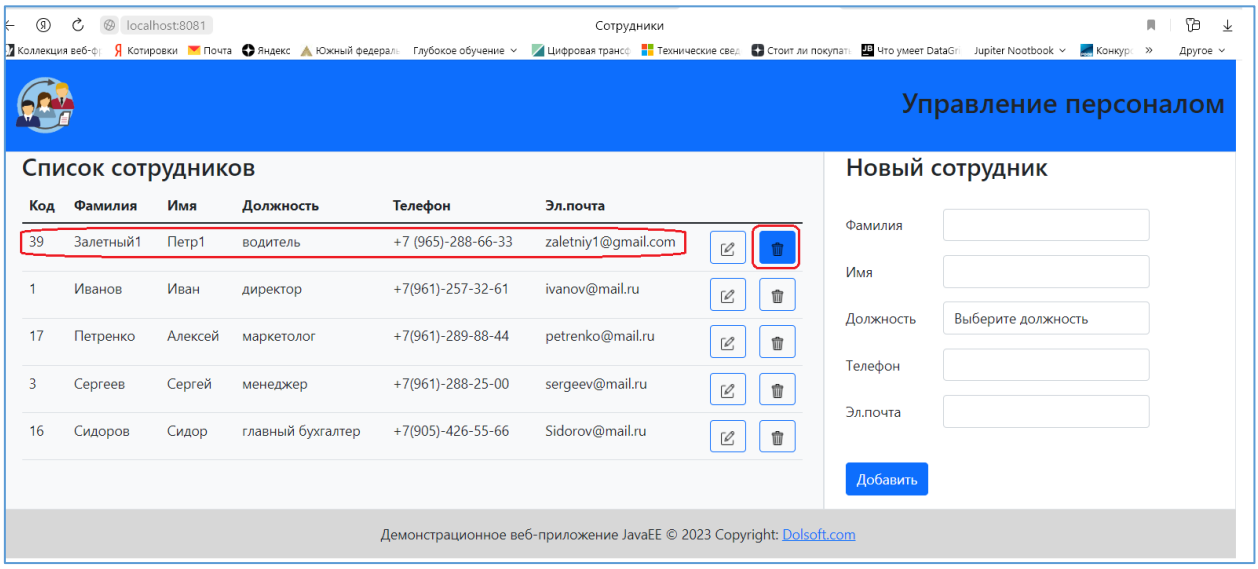


Рисунок 16 – Состояние списка сотрудников перед удалением

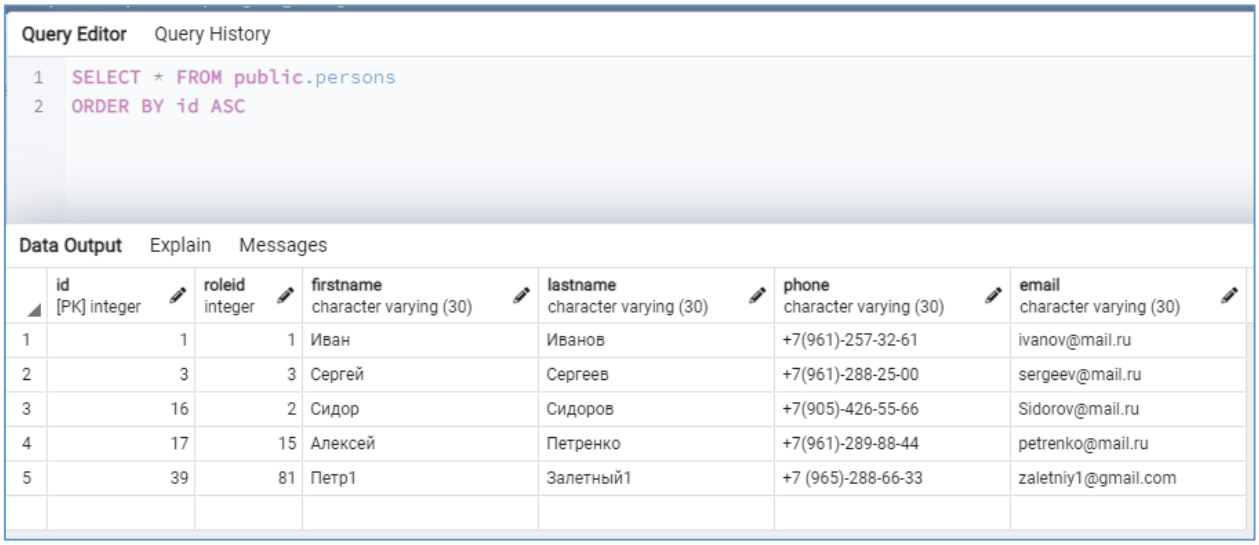


Рисунок 17 – Состояние базы данных до удаления сотруднику с кодом 39

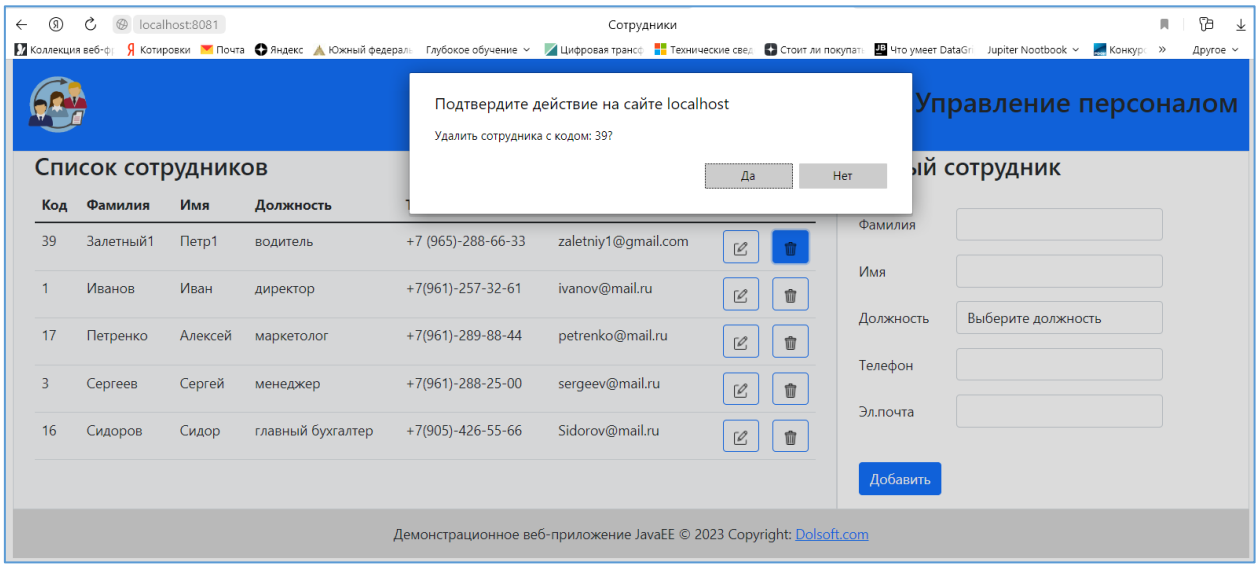


Рисунок 18 – Запрос на подтверждение удаления записи из базы данных

При нажатии кнопки «Да» сотрудник с кодом 39 будет удалена из базы данных, что отображается в списке сотрудников (рис. 19).

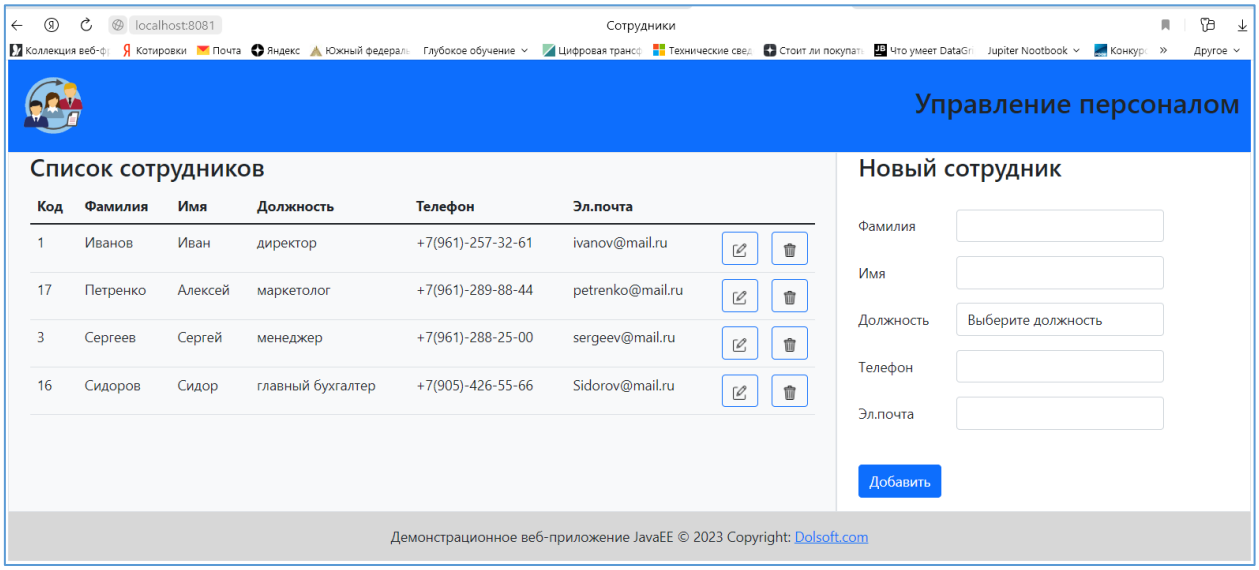
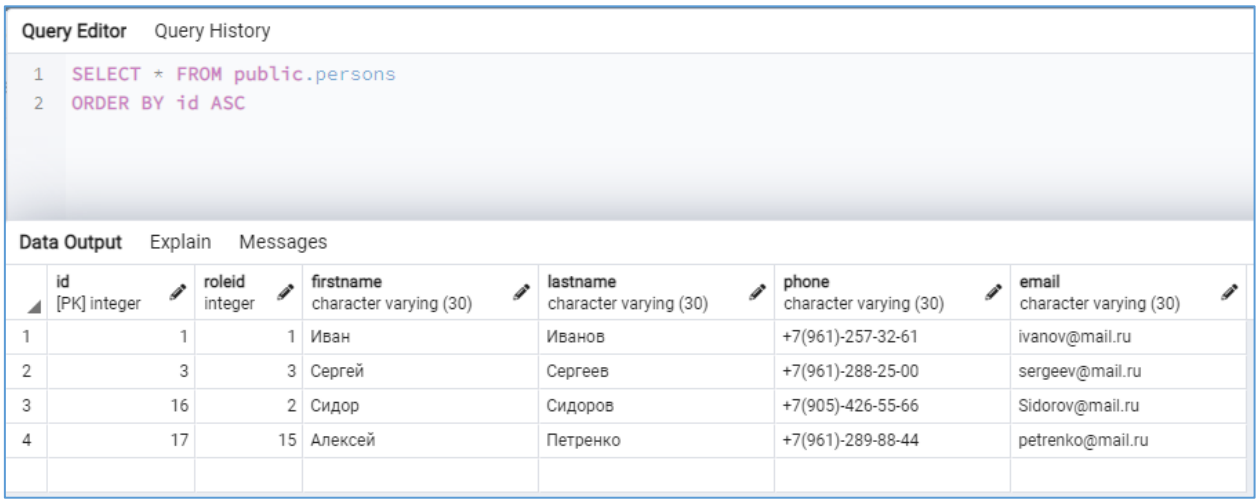


Рисунок 19 – Список сотрудников после удаления сотрудника с кодом 39

В базе данных также отображается результат удаления записи с id = 39 (рис. 20).



	id [PK] integer	roleid integer	firstname character varying (30)	lastname character varying (30)	phone character varying (30)	email character varying (30)
1	1	1	Иван	Иванов	+7(961)-257-32-61	ivanov@mail.ru
2	3	3	Сергей	Сергеев	+7(961)-288-25-00	sergeev@mail.ru
3	16	2	Сидор	Сидоров	+7(905)-426-55-66	Sidorov@mail.ru
4	17	15	Алексей	Петренко	+7(961)-289-88-44	petrenko@mail.ru

Рисунок 20 – Состояние базы данных после удаления сотрудника с кодом 39

На этом разработка функций редактирования и удаления записей в базе данных для веб-приложения завершена.

Задание на лабораторную работу

1. Изучить теоретический материал лекции 1.6 – 1.8.
2. По варианту, определенному преподавателем, разработать функционал редактирования и удаления данных в базе данных для веб-проекта и проверить работоспособность функций на тестовых данных.
3. Продемонстрировать результаты работы веб-приложения предателю.
4. Слить ветку разработки dev7 с основной веткой master.
5. Подготовить отчет по лабораторной работе и загрузить его на учебный портал.